

COIT20245 INTRODUCTION TO PROGRAMMING (HT2, 2025)

Assignment 2 - Project

Due date:	11:45 pm AEST, Sunday of Week 10 (28 th September 2025)
Weighting:	30%
Mode:	Individual
Submission:	• 1 × Python source file (.zip) • A MS-Word document (.docx) containing screenshots of testing your program, together with clear annotations
Total Marks:	30

Learning Objectives

This project draws on students' understanding of data structures, file I/O, modular programming, and control flow in relation to Python programming. The program must run in the Python console and use a menu-driven interface to guide users through available actions.

By completing this assignment, students will:

- Work with nested data structures (lists, dictionaries)
- Use functions to encapsulate logic
- Read/write CSV files for data persistence
- Implement conditional logic and loops
- Apply simple date/time operations
- Handle input validation and user feedback
- Develop a text-based user interface

These learning objectives align with Unit Learning Outcomes 1-5.

Problem Description

This individual project requires students to design and implement a console-based Library Book Catalogue & Borrowing System using Python. The program will simulate a minimal library inventory where users can manage books, search (filter) by title or author and sort book listings. It mirrors how basic library software allows library assistants to manage a simple catalogue.

In addition to managing books, the system must also support library user registration, borrowing and returning books, and handling overdue fines.

Summarised details of the basic components and the functionalities of the system are provided next.

Book Catalogue:

This will be stored and saved in a CSV file named `books.csv`. Each book in the catalogue system will have the following attributes:

- Title (string)
- Author (string)
- Year Published (integer)
- ISBN (string)
- Number of Available Copies (integer)

Library Users:

Library users are also stored and saved in a CSV file named `users.csv`. Each user is associated with the following:

- Full Name (string) : Provided at Registration
- Email (string) : Provided at Registration
- Address (string) : Provided at Registration
- Books Borrowed (a *List of Dictionaries* which contain the *Book Title* and the *Due Date*) : initially set to be empty
- Outstanding Fines (double) : initially set to be \$0.00
- User ID (string: **auto-generated** during the Registration process) as: LIB0001, LIB0002, LIB0003, ..., LIB9999. Assume that the maximum number of users to be 9999.

Borrowing Process:

This will include **due date calculation** and **inventory adjustment**. Fines must be paid in full before new books can be borrowed. You can assume the borrowing date of a book to be the *current date*. Also assume that a borrowed book must be returned within two (2) weeks.

Returning Process:

This includes **inventory adjustment** and **fine checking** for late returns. Assume that a fine of \$2 per day applies for late returns.

Project Phases

Phase-1: Implement Book Catalogue System Tasks

- Data Structure: Design the book catalogue as a *list of dictionaries* to store book records. To see how a Book Record should look like, refer to the attributes provided under Book Catalogue above.
- Core functions are outlines below:
 - `def load_catalogue(csv_filename) -> List[dict]`
Loads book records from a CSV file.
Returns the catalogue as a *list of dictionaries*.
 - `def save_catalogue(csv_filename, catalogue) -> None`
Saves the book catalogue to the same CSV file.
 - `def add_book(catalogue, title, author, year, isbn, copies)->None`
Adds the record of a new book to the catalogue.

- `def display_books(books_list) -> None`
 # Displays a list of book records (e.g., the entire catalogue or search (filtered) / sorted results)
 # to the console in a tabular format, such as in the following example.

Title	Author	Year	ISBN	Copies
Pride and Prejudice	Jane Austen	1813	978-0-19-953556-9	2
The Great Gatsby	F. Scott Fitzgerald	1925	978-0-7432-7356-5	4
Brave New World	Aldous Huxley	1932	978-0-099-53441-3	8
The Hobbit	J.R.R. Tolkien	1937	978-0-261-10221-7	4
1984	George Orwell	1949	978-0-452-28423-4	5

- `def search_by_title(catalogue, keyword) -> List[dict]`
 # Returns a list of book records whose titles match the search keyword.
- `def search_by_author(catalogue, author) -> List[dict]`
 # Returns a list of book records written by the specified author.
- `def sort_by_title(catalogue) -> None`
 # Sorts the book catalogue alphabetically by title. The original list is modified in place.
- `def sort_by_year(catalogue) -> None`
 # Sorts the book catalogue by publication year. The original list is modified in place.

Expected functionalities after Phase-1: Books can be added, displayed, searched and sorted. Book data is stored and retrieved in CSV format.

Phase-2: Implement Library User Management Tasks

- Data Structure: Manage library users as a *list of dictionaries* of user records. To see how a User Record should look like, refer to the attributes provided under Library Users above.
- Core functions are outlined below:
 - `def load_users(csv_filename) -> List[dict]`
 # Loads user records from a CSV file.
 # Returns the user records as a *list of dictionaries*.
 - `def save_users(filename, users) -> None`
 # Saves the user records to a CSV file.
 - `def register_user(users, full_name, email, address) -> str`
 # Registers a new library user and returns their unique User ID (as a string).
 # users refers to the list of library users already registered.
 # Check how many users exist when generating a new User ID.
 - `def display_user_details(users, user_id) -> None`
 # Displays details of the library user identified by a user_id.
 - `def pay_fine(users, user_id) -> None`
 # Resets the user's fine to zero and prints a confirmation.

Expected functionalities after Phase-2: New users can be registered, registered library users are given unique IDs, a user's details can be displayed, fine management is supported and user data is stored and retrieved in CSV format.

Phase-3: Tasks related to Borrowing and Returning Books

To see how the borrowing and returning processes work, refer to the details provided under the Borrowing Process and Returning Process, above.

- Core functions:
 - `def borrow_book(users, catalogue, user_id, book_title)-> None`
 # Allows a registered user with no outstanding fines to borrow a book if a copy is available.
 - `def return_book(users, catalogue, user_id, book_title, return_date)-> None`
 # Processes book return and updates inventory. Adds a fine if returned late.

Expected functionalities after Phase-3: Registered library users can borrow and return books.

Phase-4: Testing the Library Book Catalogue & Borrowing System

- You will be provided `library_interface.py`, which implements the console interface of the Library Book Catalogue & Borrowing System with the options as shown below.
- **WARNING:** You **must not modify** `library_interface.py`.

```
Library System Menu
1. Load book catalogue from CSV
2. Save book catalogue to CSV
3. Add book
4. Display all books
5. Search by title
6. Search by author
7. Sort by title
8. Sort by year
9. Load users from CSV
10. Save users to CSV
11. Register library user
12. Display library user details
13. Pay fine
14. Borrow book
15. Return book
16. Exit
Enter your choice:
```

Expected functionalities after Phase-4:

- Fully functional console interface.
- Correct output from each option.
- Robust handling of edge-cases (e.g., book not found, invalid year, etc.)

Implementation and Testing

- In addition to the assignment specification (this MS-Word document), you are provided with the following files as well, in the Assignment 2 folder on Moodle:

- (a) `library_functions.py`
- (b) `library_interface.py`
- (c) `books.csv`
- (d) `users.csv`

- (a) `library_functions.py` is the skeleton source file in which **you must write the code for all your library functions**. You **must not** change the filename. You **must** include this file in your submission.
- (b) `library_interface.py` is the Python source file that is provided to test your `library_functions.py` with the console interface. You do **not need to include** this file in your submission.
- (c) `books.csv` is an example book catalogue provided to help test your library functions. You do **not need to include** this file in your submission.
- (d) `users.csv` is an example user database provided to help test your library functions. You do **not need to include** this file in your submission.

Submission

The assignment submission must include the following files:

1. **`library_functions.py`** in a zip archive (**.zip**)
2. A MS-Word document containing screenshots of testing your program, together with clear annotations (**.docx**)

Marks will be deducted for not following these instructions. Late submission will incur a late penalty (5% of the total marks = 1.5 marks per day) as per the University's Assessment Policy and Procedure.

Marking Criteria (Maximum of 30 Marks)

Criteria	Successful implementation of the functions for:	Marks
Phase-1	• Loading the Catalogue • Saving the Catalogue • Adding a New Book • Displaying Books • Searching Books by a Title • Searching Books by an Author • Sorting Books by Title • Sorting Books by Year	1 mark × 8
Phase-2	• Loading Users' Data • Saving Users' Data • Registering a New User • Displaying User Details • Paying a Fine	2 marks × 5
Phase-3	• Borrowing a Book • Returning a Book	2 marks × 2
Phase-4	Comprehensive testing with screenshots of program execution	5 marks
Annotations	Comprehensive annotations accompanying screenshots	3 marks

Academic Integrity

Each student must write the Python programs and Annotations themselves. You may be asked to prove that you have written these items yourself. You should keep evidence that you have written them yourself, for example, early drafts of the Python code and the Annotations.

ALL assignments will be checked for plagiarism (materials copied from other students and/or material copied from other sources) using TurnItIn. If you are found to have plagiarised material or if you have used someone else's work without appropriate referencing, you will be penalised for plagiarism which could result in zero (0) marks for the whole assignment. If you falsify references/information you will also be penalised. In some circumstances, a more severe penalty may be imposed such as having a plagiarism incident raised. Please refer to CQUniversity's Policy and Procedure on Academic Integrity for details.

Once the assessment is marked, the Unit Coordinator (or his nominee) may request additional written information and/or an oral discussion to clarify the student's understanding of the submitted work. Failure to comply and/or to demonstrate an understanding of the assignment's submitted items could result in zero (0) marks for the assignment.